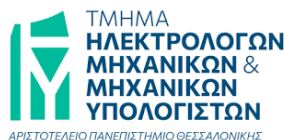


Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών



Γραφική με Υπολογιστές

Εργασία 1: Πλήρωση Τριγώνων

Τζίνα Θεοδώρα
10715
tzinatheod@ece.auth.gr

Εαρινό Εξάμηνο 2025-2026

Περιεχόμενα

1	Εισαγωγή	3
2	Γραμμική Παρεμβολή	4
2.1	Θεωρητική Ανάλυση	4
2.2	Υλοποίηση σε Python	4
3	Flat Shading	6
3.1	Θεωρητική Ανάλυση	6
3.2	Αλγόριθμος Πλήρωσης	6
3.3	Υλοποίηση σε Python	7
4	Gouraud Shading	8
4.1	Θεωρητική Ανάλυση	8
4.2	Υλοποίηση σε Python	8
4.3	Διανυσματική Υλοποίηση (Vectorization)	9
5	Shading με Texture Maps	10
5.1	Θεωρητική Ανάλυση	10
5.2	Υλοποίηση σε Python	10
5.3	Διανυσματική Υλοποίηση (Vectorization)	11
6	Χρωματισμός Αντικειμένου (Rendering)	12
6.1	Painter's Algorithm	12
6.2	Υλοποίηση σε Python	12
7	Scripts Επίδειξης (Demos)	13
7.1	Προετοιμασία Δεδομένων και Μετασχηματισμοί	13
7.2	Demo Flat Shading	13
7.3	Demo Gouraud Shading	14
7.4	Demo Texture Mapping	14
7.5	Συγκριτική Ανάλυση Μεθόδων	15

1 Εισαγωγή

Αντικείμενο της παρούσας εργασίας αποτελεί η μελέτη και η υλοποίηση βασικών αλγορίθμων της Γραφικής με Υπολογιστές για την πλήρωση και τον χρωματισμό τριγώνων σε έναν δισδιάστατο καμβά. Η διαδικασία αυτή αποτελεί τον πυρήνα της ψηφιοποίησης, όπου γεωμετρικές οντότητες μετατρέπονται σε pixels με συγκεκριμένες χρωματικές ιδιότητες. Η εργασία αναπτύχθηκε σε περιβάλλον Python, το οποίο επεξεργάζεται τρισδιάστατα δεδομένα αντικειμένων (κορυφές, ακμές και βάθος) και τα απεικονίζει χρησιμοποιώντας τρεις διαφορετικές τεχνικές σκίασης:

- **Flat Shading:** Όπου κάθε τρίγωνο χρωματίζεται ομοιόμορφα με βάση τη μέση τιμή των χρωμάτων των κορυφών του.
- **Gouraud Shading:** Μια πιο προηγμένη τεχνική όπου το χρώμα κάθε εσωτερικού σημείου υπολογίζεται μέσω γραμμικής παρεμβολής των χρωμάτων των κορυφών, προσφέροντας μια πιο λεία οπτική μετάβαση.
- **Texture Mapping:** Όπου η επιφάνεια των τριγώνων καλύπτεται από μια εξωτερική εικόνα (texture), χρησιμοποιώντας κανονικοποιημένες συντεταγμένες (u, v) για την αντιστοίχιση των σημείων.

Για την κατάλληλη απεικόνιση του αντικειμένου, εφαρμόστηκε ο **αλγόριθμος του ζωγράφου (Painter's Algorithm)**, εξασφαλίζοντας ότι τα τρίγωνα σχεδιάζονται με σειρά από το μακρινότερο προς το κοντινότερο. Η υλοποίηση βασίστηκε στη βιβλιοθήκη NumPy για τη βελτιστοποίηση των υπολογισμών και την OpenCV για τη διαχείριση και αποθήκευση των τελικών εικόνων.

2 Γραμμική Παρεμβολή

Η γραμμική παρεμβολή αποτελεί τη θεμελιώδη μαθηματική μέθοδο για την εκτίμηση τιμών σε σημεία που βρίσκονται ανάμεσα σε δύο γνωστά άκρα ενός ευθυγράμμου τμήματος. Στην παρούσα εργασία, η μέθοδος αυτή χρησιμοποιείται για τον προσδιορισμό των χρωματικών τιμών (Gouraud shading) και των συντεταγμένων υφής (texture mapping) κατά μήκος των ακμών και των οριζόντιων γραμμών σάρωσης (scanlines) των τριγώνων.

2.1 Θεωρητική Ανάλυση

Η θεωρητική βάση της παρεμβολής στηρίζεται στην υπόθεση ότι ένα σημείο p ανήκει στο ευθύγραμμο τμήμα p_1p_2 . Αν γνωρίζουμε μια διανυσματική τιμή V στα άκρα του τμήματος (V_1 στη θέση p_1 και V_2 στη θέση p_2), η τιμή στο ενδιάμεσο σημείο p υπολογίζεται ως γραμμικός συνδυασμός των V_1, V_2 .

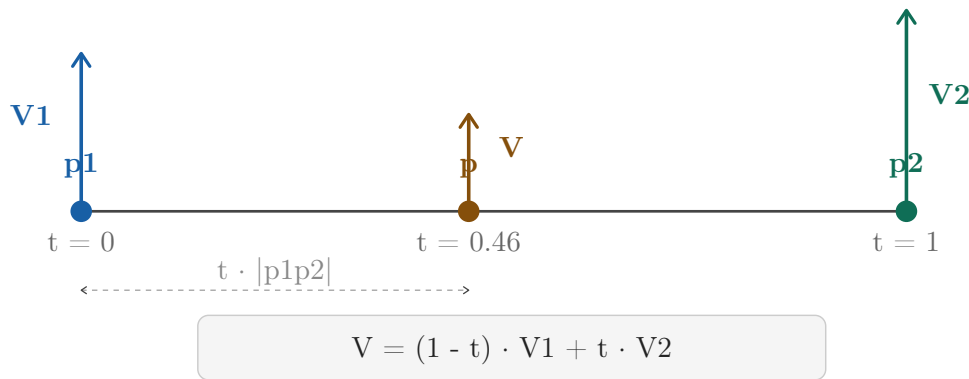


Figure 1: Γραμμική παρεμβολή διανύσματος V στο σημείο p του τμήματος p_1p_2 .

Η διαδικασία αυτή είναι κρίσιμη για τη φωτορεαλιστική απόδοση, καθώς επιτρέπει τη συνεχή μετάβαση χρωμάτων ή συντεταγμένων υφής, αποφεύγοντας τις απότομες αλλαγές που θα καθιστούσαν το αντικείμενο μη φυσικό.

2.2 Υλοποίηση σε Python

Η αντίστοιχη συνάρτηση `vector_interp` υλοποιήθηκε στο αρχείο `utils.py`, με ορίσματα εισόδου:

- p_1, p_2 : Οι δισδιάστατες συντεταγμένες των σημείων στα οποία αντιστοιχούν τα V_1, V_2 .
- V_1, V_2 : Οι διανυσματικές τιμές στις θέσεις p_1 και p_2 .
- `coord`: Η τετμημένη (x) ή η τεταγμένη (y) του σημείου p .
- `dim`: Η διάσταση ($dim = 1$ για x ή $dim = 2$ για y) στην οποία βασίζεται η παρεμβολή.

Listing 1: Συνάρτηση γραμμικής παρεμβολής

```
# If both points share the same coordinate, no interpolation needed
if np.isclose(c2 - c1, 0.0):
    t = 0.5 # Take the average
else:
    t = (coord - c1) / (c2 - c1) # Compute interpolation factor
    t = np.clip(t, 0.0, 1.0)      # Clamp t to [0, 1] to avoid extrapolation

# Perform linear interpolation
V = (1 - t) * V1 + t * V2
```

- **Διαχείριση Οριζόντιων Ακμών:** Κατά τη σάρωση ανά γραμμή (scanline), αν μια ακμή του τριγώνου είναι οριζόντια, η διαφορά $c_2 - c_1$ στον άξονα y μηδενίζεται. Η χρήση της `np.isclose` επιτρέπει στον αλγόριθμο να αναγνωρίζει ότι το σημείο δεν μπορεί να οριστεί μέσω παρεμβολής ύψους και επιστρέφει μια μέση τιμή, αποτρέποντας την κατάρρευση του μαθηματικού μοντέλου σε "εκφυλισμένες" ακμές.
- **Όρια Τριγώνου (Clamping):** Η χρήση της `np.clip` διασφαλίζει ότι οποιαδήποτε τιμή υπολογίζουμε (χρώμα ή texture) θα ανήκει αυστηρά στο εύρος που ορίζουν οι κορυφές του τριγώνου. Ακόμα και αν υπάρξει μικρό σφάλμα στρογγυλοποίησης στις συντεταγμένες των pixels, οι χρωματικές τιμές δεν θα "ξεφύγουν" ποτέ έξω από το επιτρεπτό εύρος $[0, 1]$, αποφεύγοντας οπτικά παράσιτα (artifacts) στις ακμές.
- **Παράλληλη Επεξεργασία Ιδιοτήτων:** Η διανυσματική φύση της υλοποίησης επιτρέπει στη συνάρτηση να αντιμετωπίζει το pixel όχι ως απλό σημείο, αλλά ως φορέα πολλαπλών πληροφοριών. Με μία μόνο μαθηματική πράξη, παρεμβάλλονται ταυτόχρονα όλες οι ιδιότητες (π.χ. τα τρία κανάλια RGB ή οι δύο συντεταγμένες uv).

3 Flat Shading

Η τεχνική της επίπεδης σκίασης (Flat Shading) αποτελεί την απλούστερη μέθοδο απόδοσης χρώματος σε τριγωνικά πλέγματα. Σε αυτή τη μέθοδο, κάθε τρίγωνο αντιμετωπίζεται ως μια ενιαία επιφάνεια με σταθερές χρωματικές ιδιότητες.

3.1 Θεωρητική Ανάλυση

Σύμφωνα με τις απαιτήσεις της εργασίας, όλα τα pixels που ανήκουν στο εσωτερικό του τριγώνου βάφονται με ένα ενιαίο χρώμα. Το χρώμα αυτό προκύπτει από τον διανυσματικό μέσο όρο των χρωματικών τιμών των τριών κορυφών του τριγώνου:

$$C_{flat} = \frac{1}{3} \sum_{i=1}^3 C_i$$

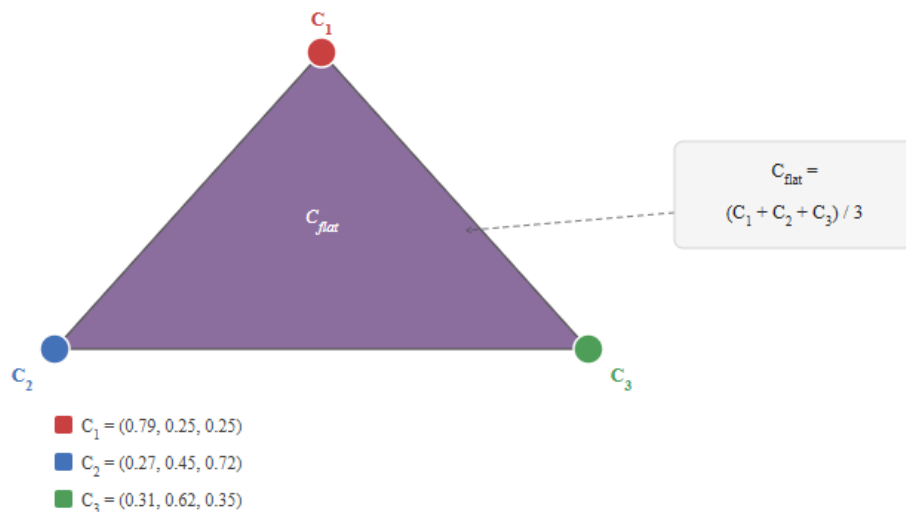


Figure 2: Απεικόνιση της μεθόδου Flat Shading

Η προσέγγιση αυτή είναι υπολογιστικά αποδοτική, αλλά παράγει μια "πολυγωνική" εμφάνιση στο τελικό αντικείμενο, καθώς οι μεταβάσεις μεταξύ γειτονικών τριγώνων είναι απότομες.

3.2 Αλγόριθμος Πλήρωσης

Για την υλοποίηση της σκίασης χρησιμοποιήθηκε ο αλγόριθμος πλήρωσης πολυγώνων (scan-line filling), προσαρμοσμένος ειδικά για την περίπτωση τριγώνων. Η διαδικασία ακολουθεί τα εξής στάδια:

1. **Εύρεση Εύρους Σάρωσης:** Προσδιορίζονται οι ακραίες τιμές y_{min} και y_{max} των κορυφών για τον ορισμό των γραμμών σάρωσης.

2. **Τομές Ακμών:** Για κάθε οριζόντια γραμμή y , υπολογίζονται τα σημεία τομής με τις ακμές του τριγώνου.
3. **Οριζόντια Πλήρωση:** Τα σημεία τομής ορίζουν ένα διάστημα $[x_{start}, x_{end}]$. Όλα τα pixels σε αυτό το διάστημα λαμβάνουν τη σταθερή τιμή C_{flat} .

3.3 Υλοποίηση σε Python

Η συνάρτηση `f_shading` στο αρχείο `shading.py` δέχεται ως είσοδο την τρέχουσα εικόνα, τις συντεταγμένες των κορυφών και τα χρώματά τους:

Listing 2: Υλοποίηση Flat Shading και Scanline Fill

```
# Compute the mean color of the triangle vertices
mean_color = np.mean(vcolors, axis=0)

# Scanline range from lowest to highest vertex
ymin = max(0, int(np.min(vy)))
ymax = min(img.shape[0] - 1, int(np.max(vy)))

for y in range(ymin, ymax + 1):
    # ... calculation of x_intersections ...

    # Paint the entire span with the mean color
    if x_start <= x_end:
        updated_img[y, x_start:x_end + 1] = mean_color
```

- **Σύμβαση Ορίων:** Για την αποφυγή διπλού χρωματισμού των κοινών ακμών μεταξύ γειτονικών τριγώνων, χρησιμοποιήθηκε η λογική του "ανοιχτού" διαστήματος στις τομές, όπου κάθε pixel ανήκει στο τρίγωνο που βρίσκεται ψηλότερα ή δεξιότερα.
- **Βελτιστοποίηση:** Η πλήρωση της γραμμής σάρωσης (`updated_img`) γίνεται μέσω slicing της NumPy, γεγονός που επιταχύνει σημαντικά τη διαδικασία σε σχέση με μια εσωτερική λούπα για κάθε pixel.

4 Gouraud Shading

Η σκίαση Gouraud (Gouraud Shading) χρησιμοποιείται για την παραγωγή ομαλών χρωματικών μεταβάσεων πάνω σε μια επιφάνεια, εξαλείφοντας την "πολυγωνική" εμφάνιση της επίπεδης σκίασης.

4.1 Θεωρητική Ανάλυση

Σε αντίθεση με το Flat Shading, η μέθοδος Gouraud υπολογίζει το χρώμα κάθε pixel μέσω διπλής γραμμικής παρεμβολής των χρωμάτων των κορυφών. Η διαδικασία χωρίζεται σε δύο φάσεις:

1. **Παρεμβολή στις Ακμές:** Για κάθε γραμμή σάρωσης y , υπολογίζονται τα χρώματα C_A και C_B στα σημεία τομής A και B της γραμμής με τις ακμές του τριγώνου.
2. **Οριζόντια Παρεμβολή:** Για κάθε pixel $P(x, y)$ ανάμεσα στα A και B , το τελικό χρώμα προκύπτει από τη γραμμική παρεμβολή μεταξύ των τιμών C_A και C_B .

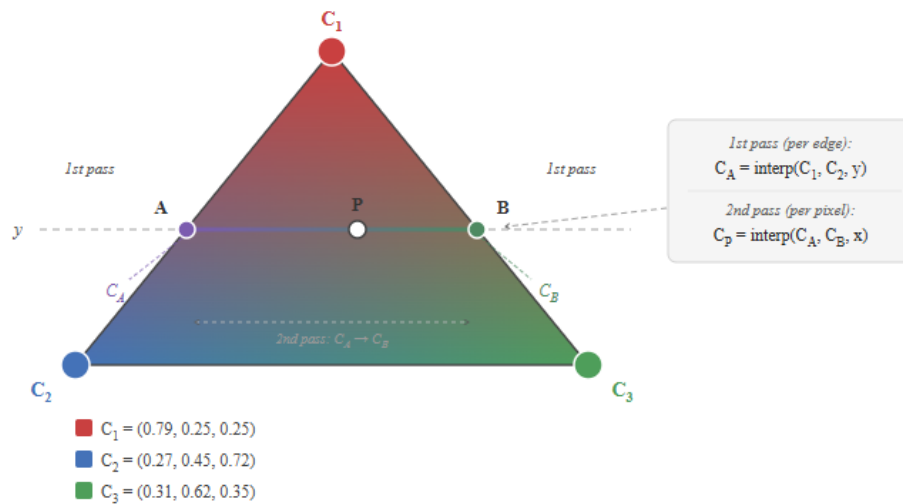


Figure 3: Απεικόνιση της μεθόδου Gouraud Shading

4.2 Υλοποίηση σε Python

Η συνάρτηση `g_shading` (στο `shading.py`) υλοποιεί την παραπάνω λογική καλώντας επαναληπτικά τη `vector_interp`:

Listing 3: Διαδικασία διπλής παρεμβολής στη σκίαση Gouraud

```
# First Pass: Interpolate color along the edge at scanline y
c_int = vector_interp(p_i, p_j, vcolors[i], vcolors[j], y, dim=2)
c_intersections.append(c_int)
```



```
# ... sorting and defining points A, B ...

# Second pass: Interpolate color horizontally from A to B for each pixel
for x in range(x_start, x_end + 1):
    color = vector_interp(p_A, p_B, c_A, c_B, x, dim=1)
    updated_img[y, x] = color
```

- **Ακρίβεια:** Η χρήση της `vector_interp` διασφαλίζει ότι τα χρώματα στις ακμές συμπίπτουν απόλυτα με τα χρώματα των γειτονικών τριγώνων, εξασφαλίζοντας οπτική συνέχεια.
- **Πολυπλοκότητα:** Η μέθοδος είναι υπολογιστικά βαρύτερη από το Flat Shading λόγω των πολλαπλών κλήσεων της συνάρτησης παρεμβολής, αλλά το οπτικό αποτέλεσμα είναι σημαντικά ανώτερο.

4.3 Διανυσματική Υλοποίηση (Vectorization)

Για τη βελτιστοποίηση της οριζόντιας πλήρωσης, υλοποιήθηκε μια εναλλακτική προσέγγιση που αντικαθιστά την εσωτερική λούπα `for x` με πράξεις πινάκων της NumPy. Σε αυτή την εκδοχή, υπολογίζονται ταυτόχρονα οι συντελεστές παρεμβολής για όλα τα pixels της γραμμής σάρωσης:

Listing 4: Διανυσματική υλοποίηση οριζόντιας παρεμβολής (Gouraud)

```
# Second pass: vectorized horizontal interpolation from A to B
xs = np.arange(x_start, x_end + 1, dtype=float)
if np.isclose(x_B - x_A, 0.0):
    t_vals = np.zeros(len(xs))
else:
    t_vals = (xs - x_A) / (x_B - x_A)
    t_vals = np.clip(t_vals, 0.0, 1.0) # Ensure t is within [0, 1]

# Calculation of color for the entire row at once
color = (1.0 - t_vals[:, None]) * c_A + t_vals[:, None] * c_B
updated_img[y, x_start:x_end + 1] = color
```

Παρόλο που η παραπάνω υλοποίηση εκμεταλλεύεται τις βελτιστοποιημένες ρουτίνες της NumPy για την επεξεργασία πινάκων, κατά τις δοκιμές **δεν παρατηρήθηκε μείωση στον συνολικό χρόνο εκτέλεσης** σε σύγκριση με την κλασική λούπα, γι αυτό παραμένει στο αρχείο πηγαίου κώδικα σε μορφή σχολίων. Η συμπεριφορά αυτή αποδίδεται στα εξής:

- **Μέγεθος Δεδομένων:** Ο αριθμός των pixels σε κάθε scanline είναι σχετικά μικρός, με αποτέλεσμα ο χρόνος που απαιτείται για τη δημιουργία των πινάκων `xs` και `t_vals` να εξισορροπεί το κέρδος από την ταχύτερη εκτέλεση της πράξης.
- **Interpreter Overhead:** Για μικρά διανύσματα, η διαφορά μεταξύ της λούπας της Python και της NumPy συχνά καλύπτεται από το overhead της κλήσης των συναρτήσεων.

5 Shading με Texture Maps

Η σκίαση με χάρτες υφής (Texture Mapping) επιτρέπει την απόδοση περίπλοκων οπτικών λεπτομερειών πάνω στην επιφάνεια των τριγώνων, "ντύνοντάς" τα με μια εξωτερική εικόνα (texture image).

5.1 Θεωρητική Ανάλυση

Η διαδικασία βασίζεται στις κανονικοποιημένες συντεταγμένες υφής $(u, v) \in [0, 1]^2$ που αντιστοιχούν σε κάθε κορυφή του τριγώνου. Η απόδοση του χρώματος ακολουθεί μια διαδικασία διπλής παρεμβολής παρόμοια με τη μέθοδο Gouraud, αλλά με στόχο τον προσδιορισμό των συντεταγμένων u και v για κάθε pixel:

1. **Παρεμβολή UV στις Ακμές:** Για κάθε γραμμή σάρωσης y , υπολογίζονται οι συντεταγμένες (u_A, v_A) και (u_B, v_B) στα σημεία τομής A και B με τις ακμές.
2. **Οριζόντια Παρεμβολή UV:** Για κάθε pixel $P(x, y)$, υπολογίζονται οι συντεταγμένες (u_P, v_P) μέσω γραμμικής παρεμβολής μεταξύ των τιμών των σημείων A και B .
3. **Δειγματοληψία (Sampling):** Οι συντεταγμένες (u_P, v_P) μετατρέπονται σε ακέραιους δείκτες (row, col) της εικόνας texture. Στην παρούσα υλοποίηση χρησιμοποιείται η μέθοδος του πλησιέστερου γείτονα (nearest neighbor) μέσω στρογγυλοποίησης.

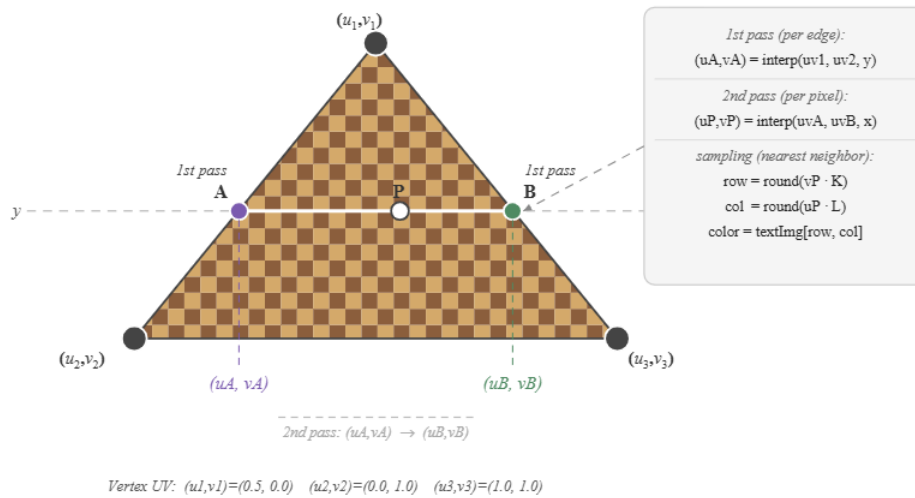


Figure 4: Απεικόνιση της μεθόδου Texture Map Shading

5.2 Υλοποίηση σε Python

Η συνάρτηση `t_shading` στο αρχείο `shading.py` διαχειρίζεται τη διαδικασία, λαμβάνοντας υπόψη τις διαστάσεις $K \times L$ της εικόνας υφής για την ορθή αντιστοίχιση των συντεταγμένων:

Listing 5: Διαδικασία παρεμβολής UV και δειγματοληψίας texture

```
# First pass: interpolate UV coordinates along the edge at scanline y
uv_int = vector_interp(p_i, p_j, uv[i], uv[j], y, dim=2)
uv_intersections.append(uv_int)

# ... calculation of points A and B ...

# Second pass: interpolate UV horizontally from A to B for each pixel
for x in range(x_start, x_end + 1):
    uv_p = vector_interp(p_A, p_B, uv_A, uv_B, x, dim=1)

    # Map normalized UV coordinates to texture image pixel indices
    tex_col = np.round(uv_p[0] * L).astype(int) % L
    tex_row = np.round(uv_p[1] * K).astype(int) % K

    # Look up and copy the color from the texture image
    updated_img[y, x] = textImg[tex_row, tex_col]
```

- **Δειγματοληψία Πλησιέστερου Γείτονα:** Για την ανάκτηση χρώματος από μη ακέραιες θέσεις της εικόνας texture, επιλέχθηκε η τιμή της πλησιέστερης ακέραιας θέσης μέσω της εντολής `np.round`.
- **Διαχείριση Ορίων:** Χρησιμοποιήθηκε ο τελεστής modulo (%) κατά τη μετατροπή των UV σε δείκτες πινάκων, ώστε αν μια παρεμβολή δώσει τιμή οριακά εκτός του $[0, 1]$, ο αλγόριθμος να "ανακυκλώσει" την υφή (αποφυγή index out of bounds).

5.3 Διανυσματική Υλοποίηση (Vectorization)

Όπως και στη σκίαση Gouraud, υλοποιήθηκε μια διανυσματική εκδοχή οριζόντιας παρεμβολής των συντεταγμένων UV και της μαζικής δειγματοληψίας χρωμάτων της εικόνας υφής:

Listing 6: Διανυσματική υλοποίηση texture mapping

```
# Second pass: vectorized horizontal UV interpolation from A to B
xs = np.arange(x_start, x_end + 1, dtype=float)
if np.isclose(x_B - x_A, 0.0):
    t_vals = np.zeros(len(xs))
else:
    t_vals = (xs - x_A) / (x_B - x_A)
    t_vals = np.clip(t_vals, 0.0, 1.0)

uvs_row = (1.0 - t_vals[:, None]) * uv_A + t_vals[:, None] * uv_B # (N, 2)

tex_cols = np.round(uvs_row[:, 0] * L).astype(int) % L
tex_rows = np.round(uvs_row[:, 1] * K).astype(int) % K

updated_img[y, x_start:x_end + 1] = textImg[tex_rows, tex_cols]
```

Η διανυσματική υλοποίηση της `t_shading`, ομοίως με της `g_shading`, δεν παρουσίασε βελτίωση στον χρόνο εκτέλεσης σε σχέση με την υλοποίηση μέσω λούπας.

6 Χρωματισμός Αντικειμένου (Rendering)

Η συνάρτηση `render_img` αποτελεί το κεντρικό σημείο ελέγχου του συστήματος, καθώς είναι υπεύθυνη για τη σύνθεση της τελικής εικόνας από το σύνολο των τριγώνων που ορίζουν το αντικείμενο.

6.1 Painter's Algorithm

Για τη σωστή διαχείριση της πλήρωσης χρώματος και την επίλυση του προβλήματος των κρυφών επιφανειών, υλοποιήθηκε ο **Αλγόριθμος του Ζωγράφου (Painter's Algorithm)**. Σύμφωνα με αυτόν, τα τρίγωνα δεν σχεδιάζονται με τυχαία σειρά, αλλά με βάση την απόστασή τους από τον παρατηρητή (βάθος):

1. **Υπολογισμός Βάθους:** Για κάθε τρίγωνο, υπολογίζεται το βάθος του ως ο αριθμητικός μέσος όρος (κέντρο βάρους) του βάθους των τριών κορυφών του.
2. **Ταξινόμηση:** Τα τρίγωνα ταξινομούνται σε φθίνουσα σειρά βάσει του βάθους τους (από το μεγαλύτερο προς το μικρότερο).
3. **Σχεδίαση (Back-to-Front):** Η διαδικασία πλήρωσης ξεκινά από τα μακρινότερα τρίγωνα και καταλήγει στα κοντινότερα. Με αυτόν τον τρόπο, τα αντικείμενα που βρίσκονται "μπροστά" επικαλύπτουν σωστά αυτά που βρίσκονται "πίσω".

6.2 Υλοποίηση σε Python

Η συνάρτηση αρχικοποιεί έναν λευκό καμβά διαστάσεων 512×512 και εκτελεί την ταξινόμηση και τη διαδοχική κλήση των ρουτινών σκίασης:

Listing 7: Η συνάρτηση `render_img` και η διαχείριση βάθους

```
# Compute the depth of each triangle as the mean of its vertices
triangle_depths = np.mean(depth[faces], axis=1)

# Sort triangles by depth in descending order (farthest first)
sorted_indices = np.argsort(triangle_depths)[::-1]

for idx in sorted_indices:
    # Call the appropriate shading routine
    if shading == "f":
        img = f_shading(img, tri_vertices, tri_vcolors)
    elif ...
```

- **Διαχείριση Μνήμης:** Η εικόνα `img` περνάει διαδοχικά από κάθε κλήση σκίασης, με την κάθε συνάρτηση (`f_shading`, `g_shading`, `t_shading`) να επιστρέφει έναν ενημερωμένο πίνακα, διατηρώντας τα προηγούμενα τρίγωνα στο παρασκήνιο.
- **Αρχικοποίηση:** Η χρήση της `np.ones` για τη δημιουργία του καμβά εξασφαλίζει ότι το υπόβαθρο της σκηνής είναι απόλυτα λευκό ($RGB = 1, 1, 1$), όπως ορίζεται στις οδηγίες.

7 Scripts Επίδειξης (Demos)

Για την αξιολόγηση των συναρτήσεων σκίασης, δημιουργήθηκαν τρία scripts επίδειξης τα οποία υλοποιούν την πλήρη ροή εργασίας, από τη φόρτωση των δεδομένων έως την παραγωγή και αποθήκευση της τελικής εικόνας.

7.1 Προετοιμασία Δεδομένων και Μετασχηματισμοί

Και τα τρία scripts ακολουθούν μια κοινή διαδικασία προετοιμασίας των δεδομένων εισόδου, τα οποία αντλούνται από το αρχείο `hw1.npy`. Κατά τη φόρτωση, πραγματοποιήθηκαν οι εξής απαραίτητοι γεωμετρικοί μετασχηματισμοί:

- **Αναστροφή Αξόνων (Flipping):** Οι συντεταγμένες των κορυφών υπέστησαν αναστροφή τόσο στον άξονα x ($511 - x$) όσο και στον άξονα y ($511 - y$).
- **Αιτιολόγηση:** Οι αναστροφές αυτές κρίθηκαν απαραίτητες ώστε η προβολή του αντικειμένου να συμφωνεί με τον αναμενόμενο προσανατολισμό του καμβά και το σύστημα συντεταγμένων της βιβλιοθήκης OpenCV.

7.2 Demo Flat Shading

Το script `demo_f.py` φορτώνει τις 2D θέσεις των κορυφών, τα χρώματα και τη συνδεσιμότητα των τριγώνων (faces).

- **Λειτουργία:** Καλεί τη `render_img` με την παράμετρο `shading='f'`.
- **Αποτέλεσμα:** Παράγει την εικόνα `output_flat.png`, όπου το αντικείμενο εμφανίζεται με έντονο πολυγωνικό χαρακτήρα λόγω του ομοιόμορφου χρωματισμού κάθε τριγώνου.

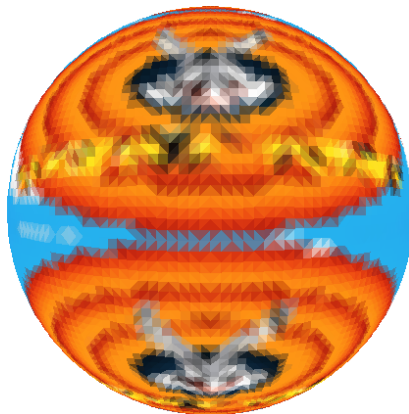


Figure 5: Αποτέλεσμα επίπεδης σκίασης (Flat Shading).

7.3 Demo Gouraud Shading

Στο script `demo_g.py`, η διαδικασία διαφοροποιείται ως προς τη μέθοδο κλήσης της μηχανής απόδοσης.

- **Λειτουργία:** Καλεί τη `render_img` με την παράμετρο `shading='g'`.
- **Αποτέλεσμα:** Παράγει την εικόνα `output_gouraud.png`. Η χρήση της γραμμικής παρεμβολής χρωμάτων προσδίδει στο αντικείμενο μια συνεχή και λεία εμφάνιση, εξαλείφοντας τις ορατές ακμές μεταξύ των τριγώνων.

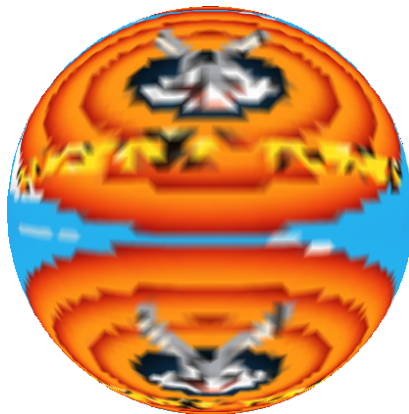


Figure 6: Αποτέλεσμα σκίασης Gouraud.

7.4 Demo Texture Mapping

Το script `demo_t.py` είναι το πιο σύνθετο, καθώς απαιτεί τη διαχείριση επιπλέον δεδομένων υφής.

- **Διαχείριση UV:** Πραγματοποιείται αναστροφή στον άξονα u των συντεταγμένων υφής (`1.0 - uvs[:, 1]`) ώστε να ευθυγραμμιστούν με την αναστροφή του άξονα y που έγινε στις κορυφές.
- **Φόρτωση Υφής:** Εισάγεται η εικόνα `loony-repeat.png`, η οποία μετατρέπεται σε μορφή RGB και κανονικοποιείται στο διάστημα $[0, 1]$.
- **Λειτουργία:** Καλεί τη `render_img` με την παράμετρο `shading='t'` και το αντικείμενο `texImg`.
- **Αποτέλεσμα:** Παράγει την εικόνα `output_texture.png`, όπου το τρισδιάστατο μοντέλο εμφανίζεται "ντυμένο" με την επιλεγμένη υφή.



Figure 7: Η εικόνα που χρησιμοποιήθηκε ως texture map



Figure 8: Αποτέλεσμα σκίασης με Texture Mapping.

7.5 Συγκριτική Ανάλυση Μεθόδων

Στον παρακάτω πίνακα συνοψίζονται οι βασικές διαφορές μεταξύ των τεχνικών που υλοποιήθηκαν:

Μέθοδος	Flat Shading	Gouraud Shading	Texture Mapping
Ποιότητα	Χαμηλή	Υψηλή	Πολύ Υψηλή
Υπολ. Κόστος	Ελάχιστο	Μέτριο	Υψηλότερο
Λειτουργία	Μέσος όρος χρωμάτων	Παρεμβολή χρωμάτων	Παρεμβολή UV & Sampling

Table 1: Σύγκριση των μεθόδων σκίασης που αναπτύχθηκαν.